GENETIC ALGORITHMS – IMAGE APPROXIMATION

Artificial Intelligence – TP2

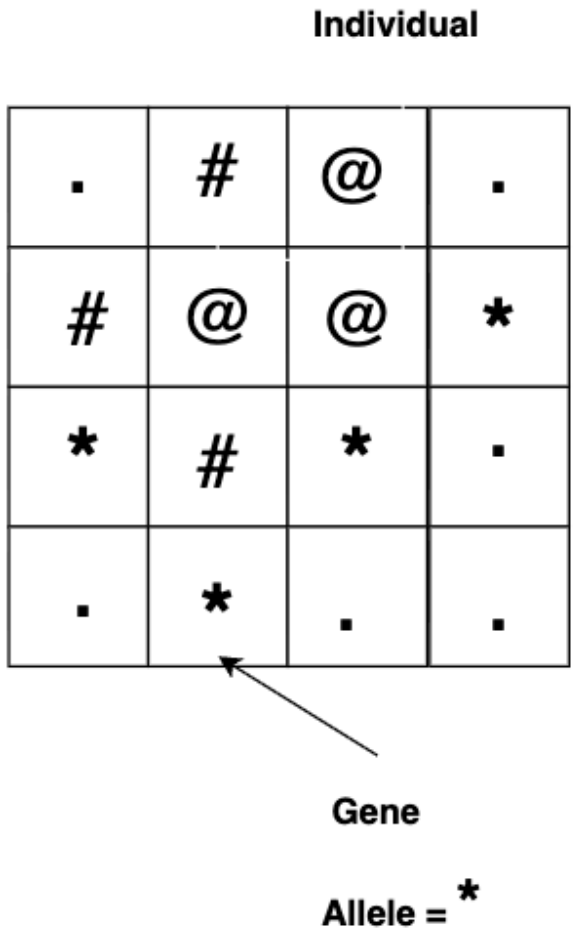
Group 13: Albertine Gjøs, Hannah Hauge Kalager, Stella Figueirado Kirkvaag, Marek Myrebøe-Engels, Mika Van Kempen, Priska Viljakainen

EXERCISE 1 – ASCII ART

Design a Genetic Algorithm that approximates a square target image as an N x N ASCII character grid.

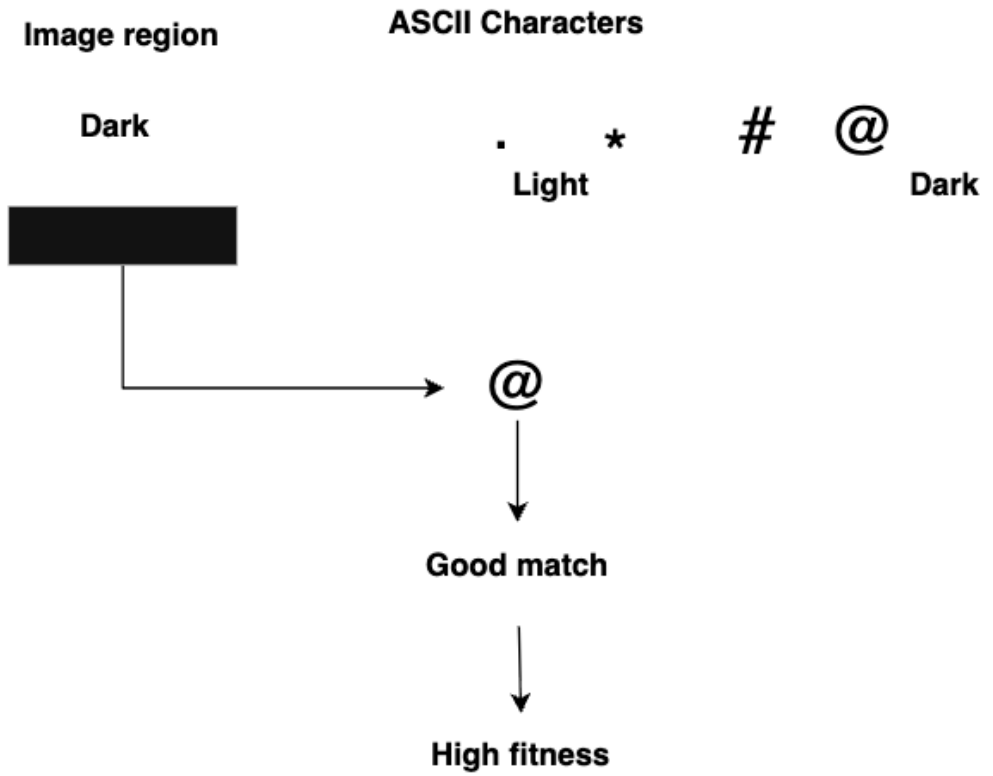
REPRESENTATION

- Individual: a complete N x N ASCII grid representing one candidate solution
- Gene: one cell in the grid
- Chromosome: all cells (genes) that make up the individual
- Allele: the ASCII character assigned to that cell



FITNESS

- Divide the target image into N × N regions.
- Calculate the average grayscale brightness of each region.
- Compare the brightness with the visual density of the corresponding ASCII character.
- Higher similarity ⟶ Higher fitness

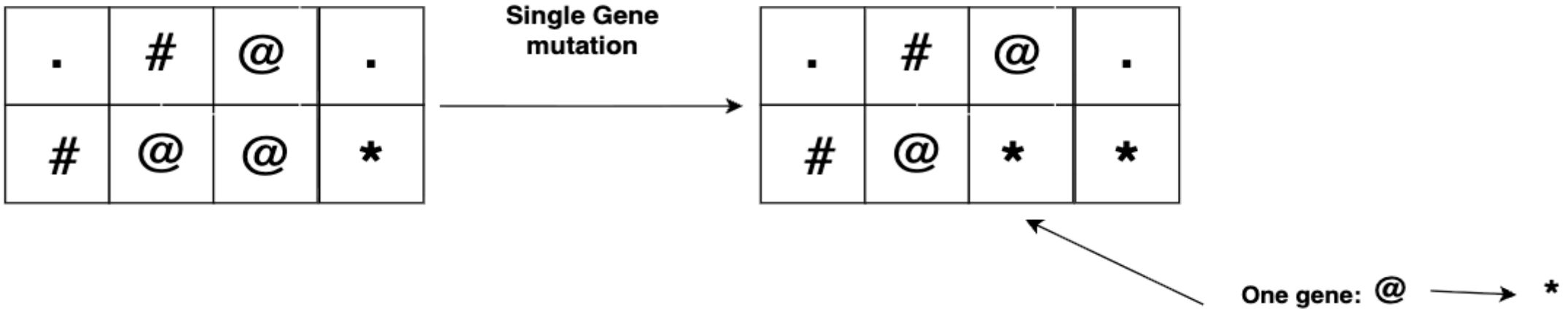


PARENT SELECTION

- **Purpose:** Determines which candidate solutions are chosen as parents for the next generation.
- Selection methods vary from those that strictly choose the best-fitting parents to those that take randomness into account as well.
- To avoid premature convergence, a good fit for the exercise could be for example tournament-based selection methods.

MUTATION

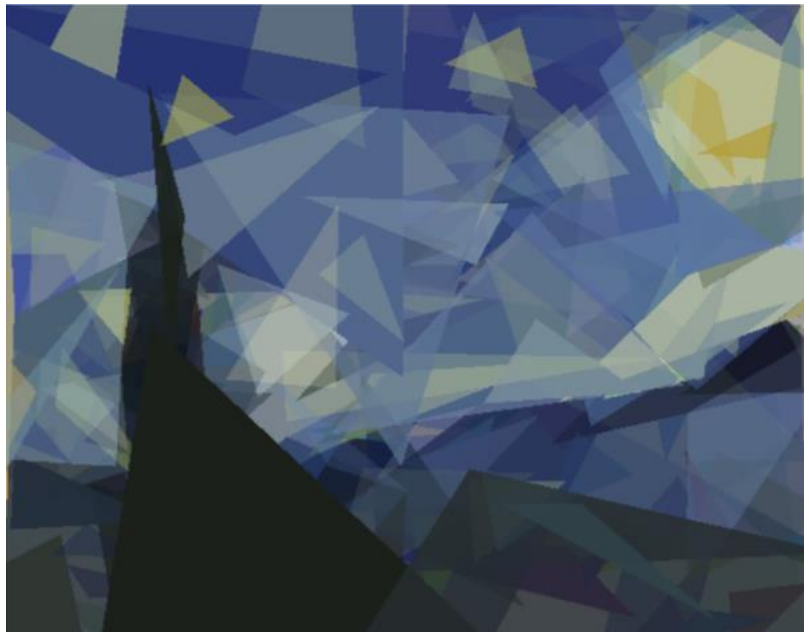
- Mutation randomly modifies one or more characters in the ASCII grid.
- Possible approaches
  - **Single-gene:** Replace one ASCII character
  - **Multigene:** Replace multiple ASCII characters
  - **Region-based:** Mutate several characters within a small region
- Purpose
  - Introduce diversity and help prevent premature convergence.



Marek

EXERCISE 2 – IMAGE APPROXIMATION USING TRIANGLES

Approximate a target image using a fixed number of semi-transparent, uniformly colored triangles.



EXERCISE 2 – IMAGE APPROXIMATION USING TRIANGLES

Marek

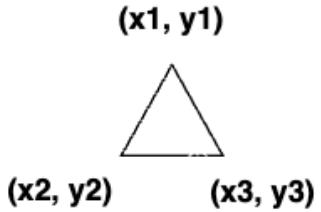
- Input:
  - Target image
  - Number of triangles
  - Genetic Algorithm hyperparameters
- Output:
  - Best generated image
  - Triangle information (position, color, transparency)
    - Each triangle has a uniform color and can be translucent (RGBA).
  - Performance metrics (fitness, error, generations) score = 1/(1+k*error)
- Optimization: Evolve triangle configurations using a genetic algorithm

Albertine

SOLUTION REPRESENTATION

- Gene: One translucent triangle defined by
  - three vertices (x_1, y_1), (x_2, y_2), (x_3, y_3)
  - color and transparency (R, G, B, A)
- Individual: A fixed-size list of triangles representing one candidate image
- Population: A collection of candidate images
- **Why this representation?** Each triangle can be modified independently, providing a simple representation that the GA can evolve.

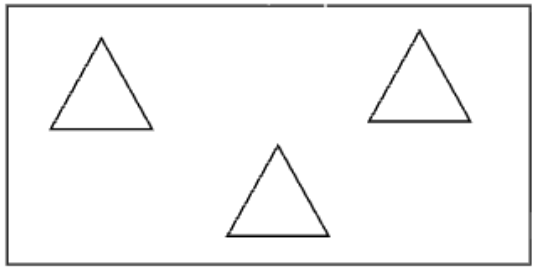
Gene



One triangle

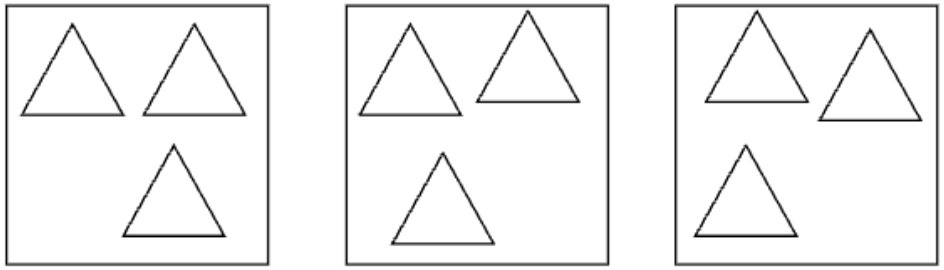
Position & Shape: 3 vertices
Appearance: RGBA (color + transparency)

Individual



One candidate image

Population

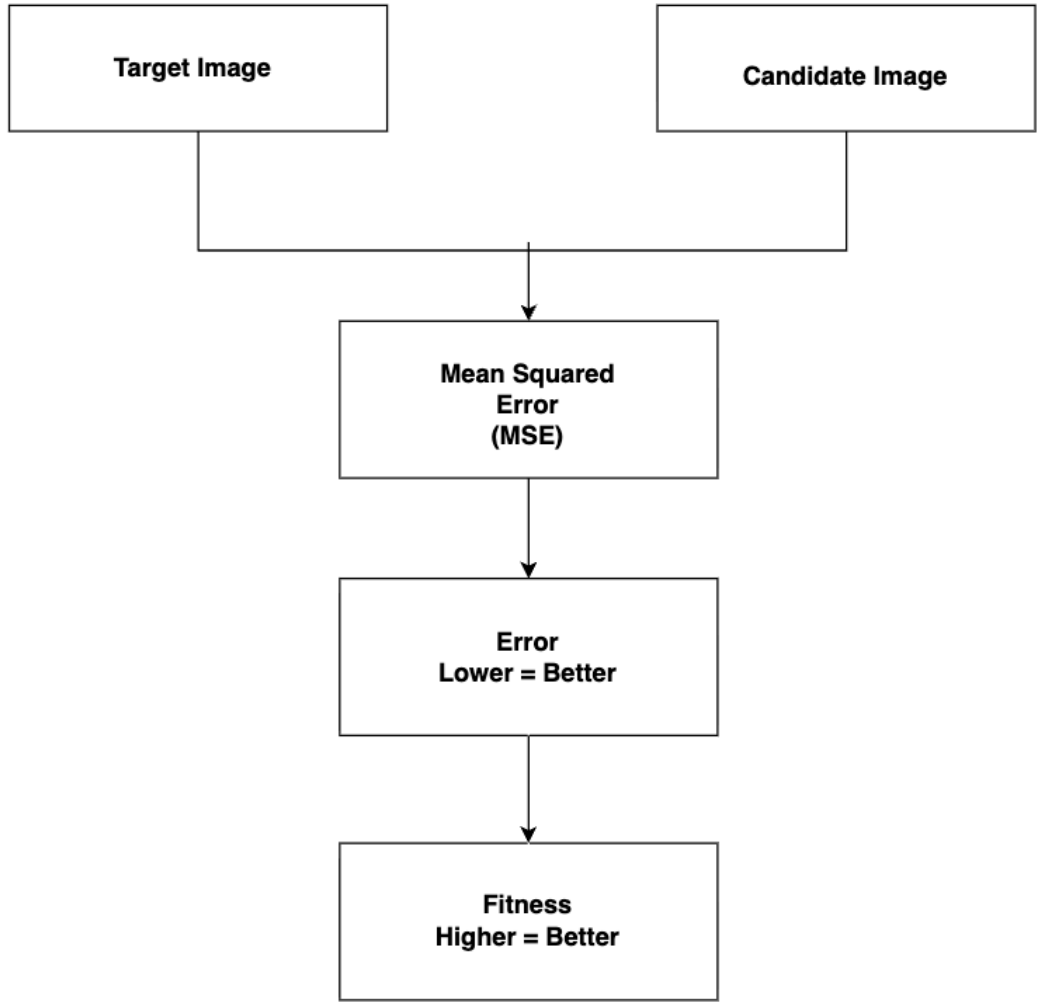


Multiple candidates

FITNESS FUNCTION

Marek

- The fitness function measures how closely a candidate image resembles the target image.
- Image difference is calculated using Mean Squared Error (MSE).
- Lower error → Higher fitness → Better individual
- Why MSE?
    - Simple and efficient measure of pixel-level similarity.



SELECTION METHODS

Selection determines which individuals are chosen as parents to produce the next generation.

- Elite
  - Selects the individuals with the highest fitness.
- Roulette
  - Individuals with higher fitness have a higher probability of being selected.
- Universal
  - Similar to Roulette, but uses evenly spaced selection points.
- Boltzmann
  - Uses temperature to control the selection pressure.
- Deterministic Tournament
  - Random individuals compete and the one with the highest fitness is selected.
- Probabilistic Tournament
  - The best individual usually wins, but weaker individuals can also be selected.
- Ranking
  - Selection probability is based on the individual's rank rather than its raw fitness.

CROSSOVER & MUTATION

CROSSOVER

- Combines genetic information from two parent individuals to create offspring.
- One-point Crossover
  - A single crossover point is chosen, and the offspring inherits one part from each parent
- Two-point Crossover
  - Two crossover points are chosen, and the section between them is exchanged between the parents.
- Uniform Crossover
  - Each gene is independently chosen from either parent, usually with equal probability.

Marek

MUTATION

- Introduces random changes to maintain diversity and explore new solutions.
- Gene
  - With a given probability, mutates one randomly selected triangle.
- Multigene
  - Randomly selects several triangles, then mutates each selected triangle with the mutation probability.
- Uniform
  - Checks every triangle independently and mutates it with the mutation probability.
- Complete
  - With the mutation probability, mutates every triangle in the individual.

Albertine

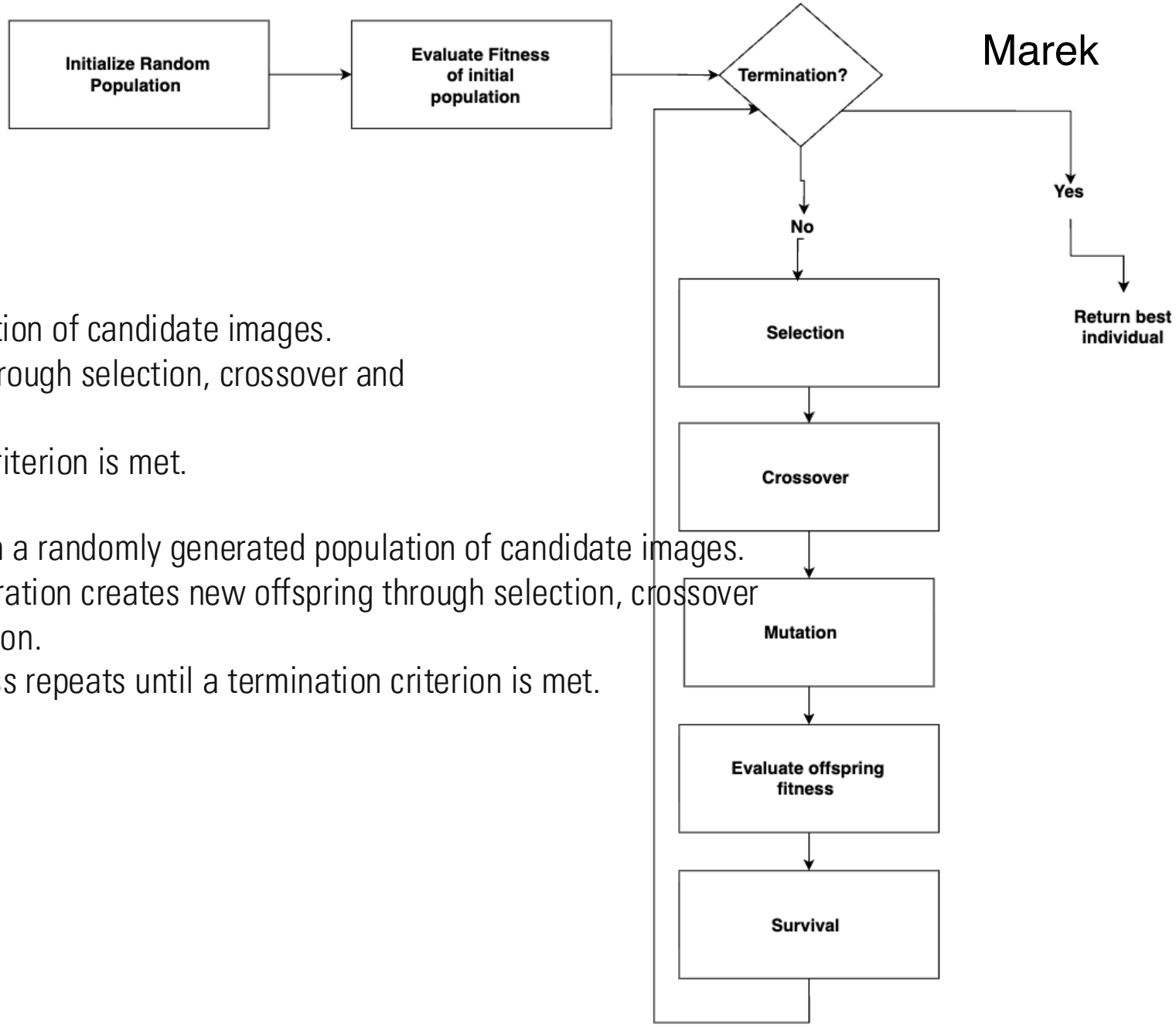
GENETIC ALGORITHM FLOW

HOW IT WORKS

How it works

- Starts with a randomly generated population of candidate images.
- Each generation creates new offspring through selection, crossover and mutation.
- The process repeats until a termination criterion is met.

- Starts with a randomly generated population of candidate images.
- Each generation creates new offspring through selection, crossover and mutation.
- The process repeats until a termination criterion is met.



SURVIVAL & TERMINATION

SURVIVAL

Survival determines which individuals form the next generation.

- Additive (μ + λ)
  - Parents and offspring compete for survival
  - The best individuals survive

- Exclusive (μ, λ)
  - Only offspring compete
  - Parents are discarded

TERMINATION

The algorithm stops when one of the termination criteria is met.

- Maximum generations
  - Stops after the configured number of generations
- Minimum error
  - Stops when the best image reaches the configured error threshold
  - Only used when **`min_error`** is specified

DESIGN CHOICES & JUSTIFICATION

- Representation - Triangles Albertine
  - Simple representation using position, color and transparency
  - Easy to modify through crossover and mutation
  - More triangles allow more detail, but also increase the search space
- Fitness - MSE Marek
  - Compares the generated image with the target pixel by pixel
  - Simple and fast to calculate
  - Important since fitness is evaluated many times during a run
- Crossover & Mutation Albertine
  - Different methods introduce different amounts of variation
  - We implemented multiple methods and compare them experimentally
  - This allows us to choose based on results rather than assuming one method is best
- Termination Marek
  - Maximum generations gives us a clear limit on computation
  - Minimum error allows the algorithm to stop early when the result is good enough

EXPERIMENTS & RESULTS

1. Quality: How good is the final approximation?
   - Final MSE
2. Speed: How quickly does the algorithm improve?
   - Convergence over generations
   - Runtime
3. Stability: How consistent are the results?
   - Variation across independent runs (IQR)

EXPERIMENT 1 — SELECTION

EXPERIMENT 2 — CROSSOVER

EXPERIMENT 3 — MUTATION

EXPERIMENT 4 — NUMBER OF TRIANGLES

EXPERIMENT 5 — POPULATION SIZE

OVERALL COMPARISON

EXPERIMENT 6 — SURVIVAL STRATEGY

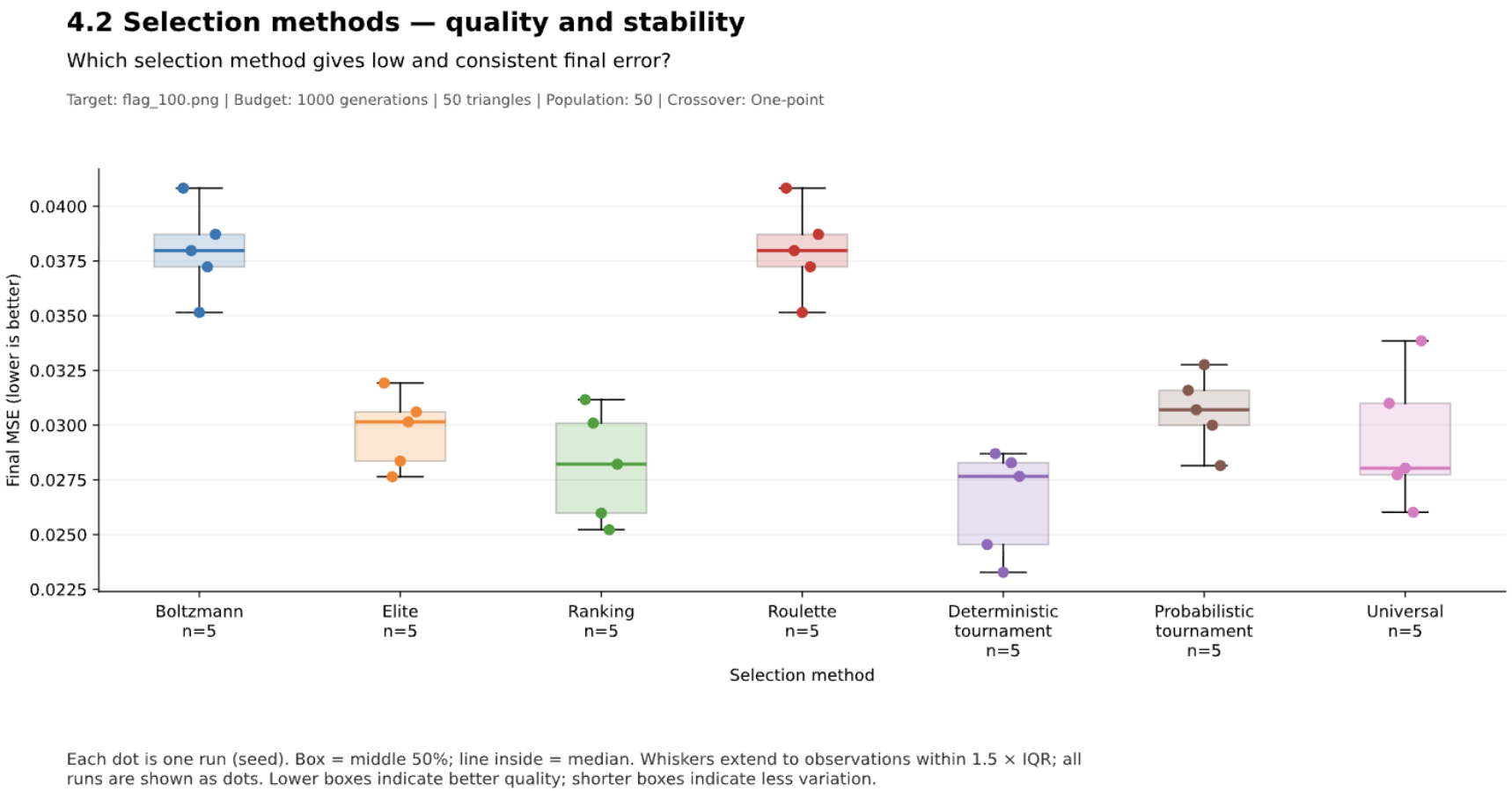
OVERALL RESULTS

Baseline:
- Triangles: 50
- Population: 50
- Generations: 1000
- Selection: Elite
- Crossover: One-point
- Mutation: Gene
- Survival: Additive

5 independent runs per configuration · Seeds 1–5 · One parameter changed at a time

SELECTION METHODS

How does the selection strategy affect final quality and stability?



Best median MSE: Deterministic Tournament — 0.0277
Universal — 0.0280
Ranking — 0.0282

Deterministic Tournament achieved the lowest median MSE, although Universal and Ranking performed similarly.

CROSSOVER METHODS

How does crossover strategy affect final quality and stability?

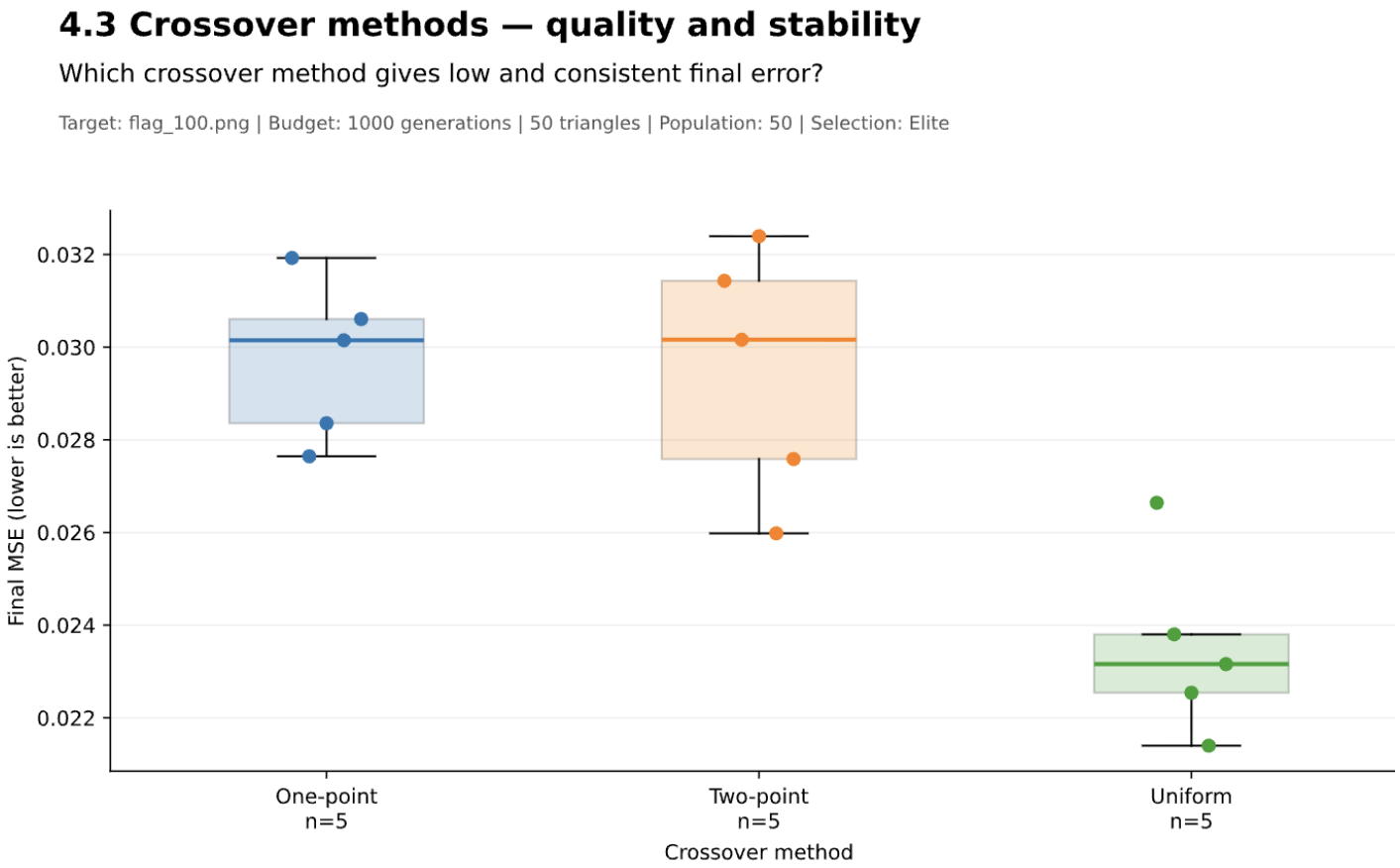
Comparing: One-point / Two-point / Uniform

Uniform: 0.0232

One-point: 0.0301

Two-point: 0.0302

Conclusion: Uniform crossover achieved the lowest median MSE and the lowest run-to-run variation.
Independent triangle inheritance may combine useful elements from both parents more effectively.

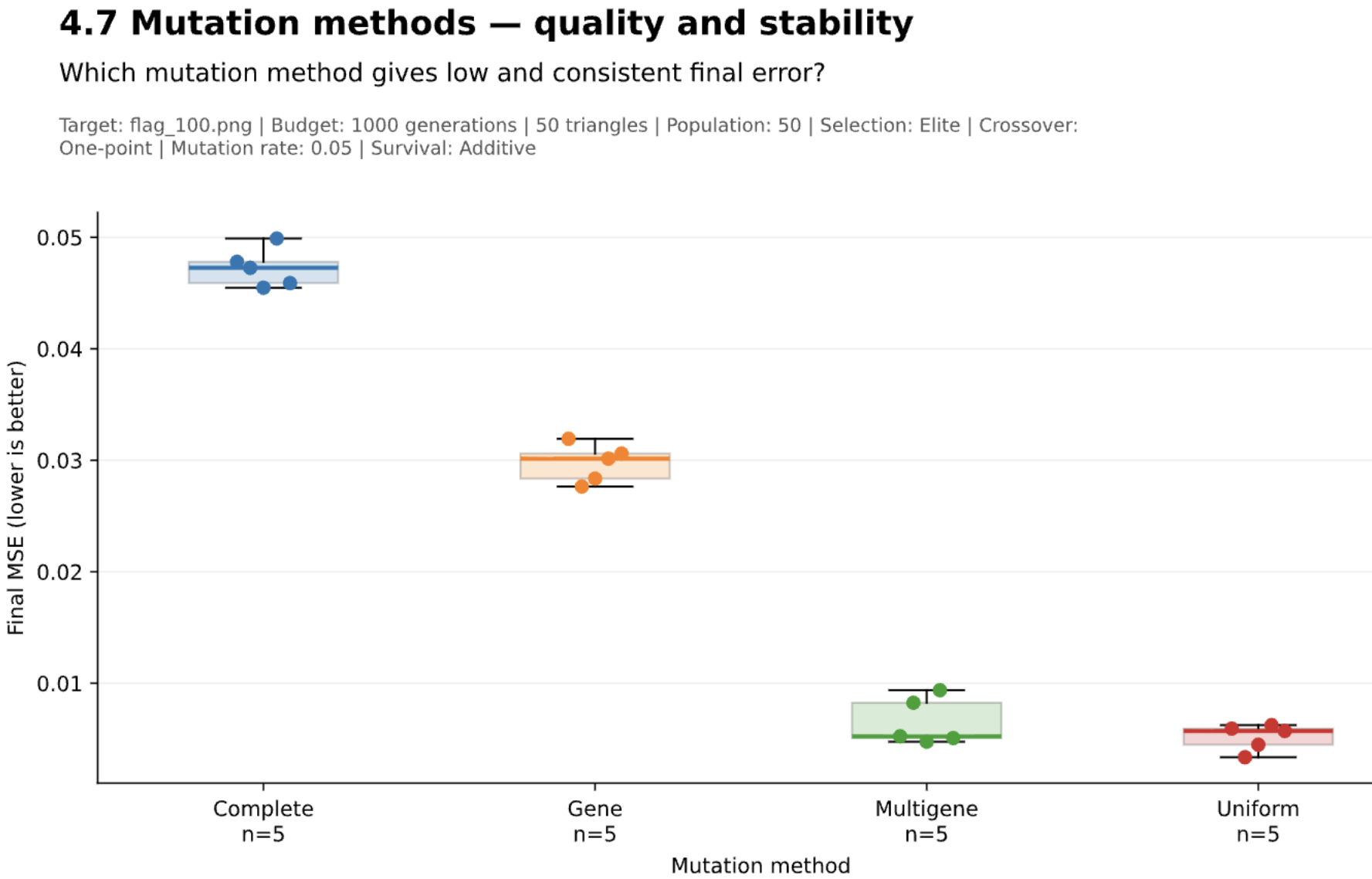


Each dot is one run (seed). Box = middle 50%; line inside = median. Whiskers extend to observations within 1.5 × IQR; all runs are shown as dots. Lower boxes indicate better quality; shorter boxes indicate less variation.

MUTATION METHODS

How does mutation strategy affect approximation quality?

- Multigene — 0.0052
  Uniform — 0.0057
  Gene — 0.0302
  Complete — 0.0473
- Conclusion: Multigene and Uniform mutation substantially outperformed Gene and Complete mutation, with almost no increase in runtime.

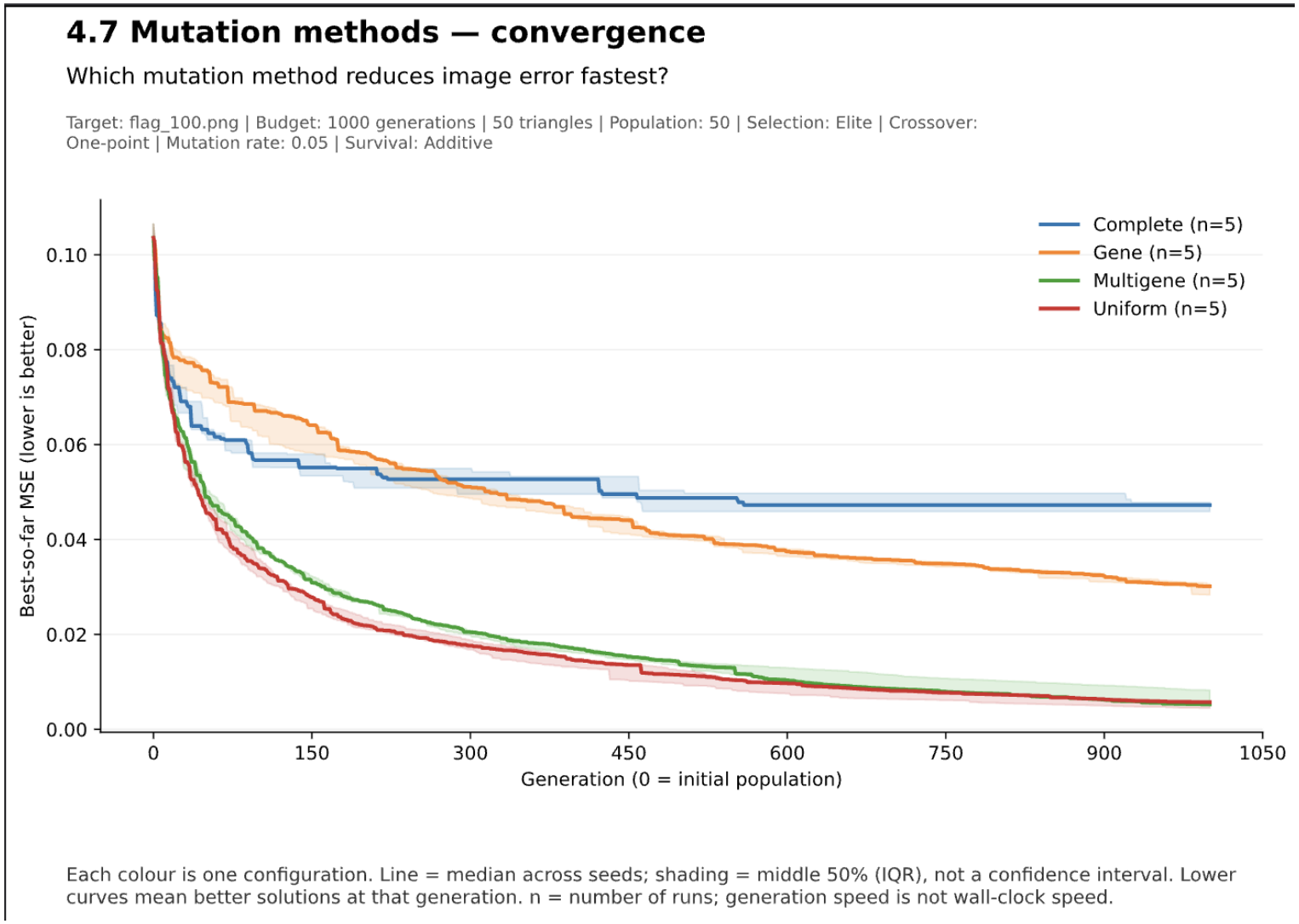


Each dot is one run (seed). Box = middle 50%; line inside = median. Whiskers extend to observations within 1.5 × IQR; all runs are shown as dots. Lower boxes indicate better quality; shorter boxes indicate less variation.

MUTATION METHODS - CONVERGENCE ANALYSIS

How quickly does the algorithm improve over generations?

- Multigene and Uniform improve faster than Gene and Complete.
- Complete stops improving early, while Multigene and Uniform continue improving.
- Mutation method has a clear effect on convergence.

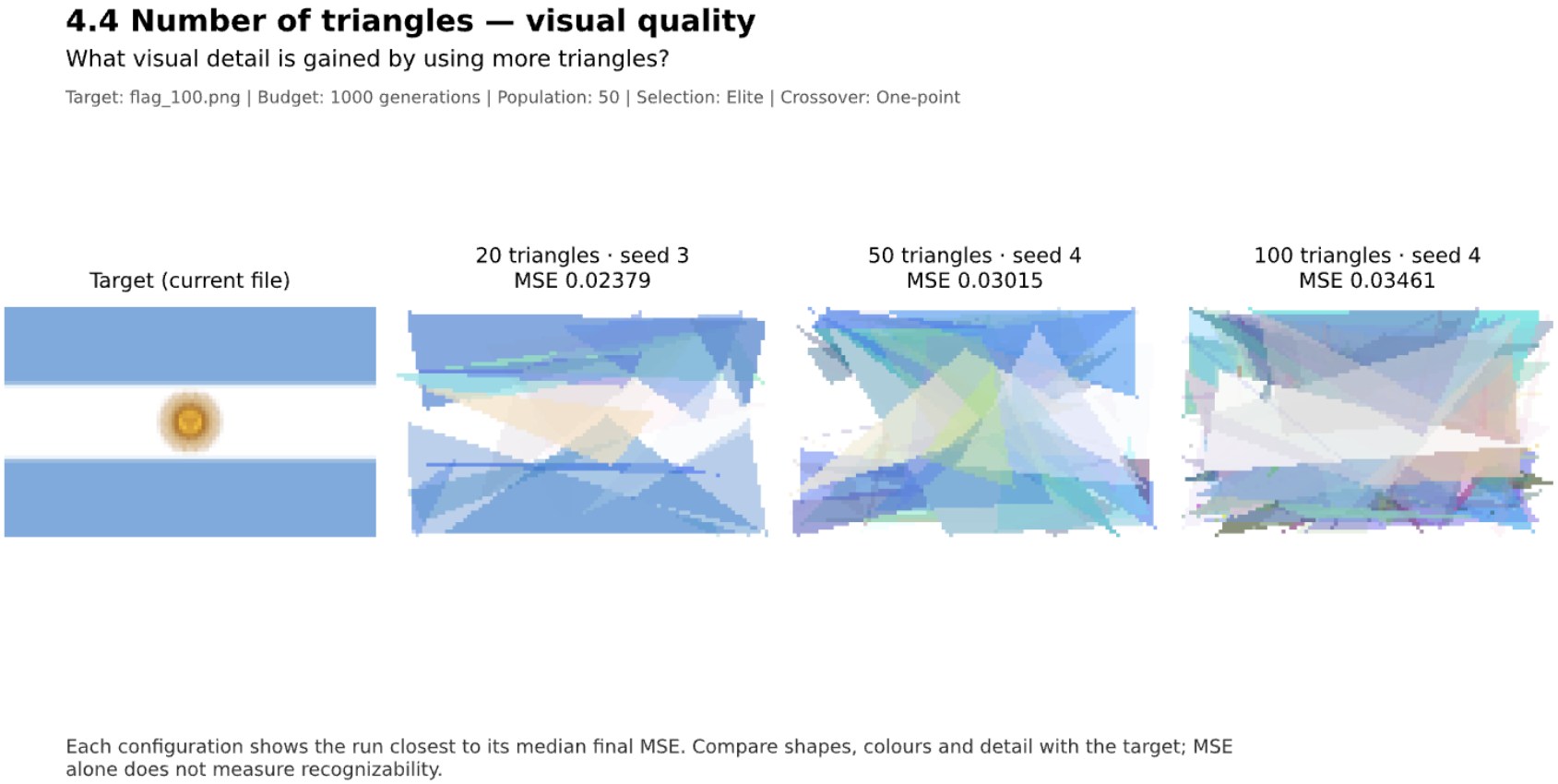


NUMBER OF TRIANGLES

Does a more expressive representation improve the approximation under a fixed search budget?

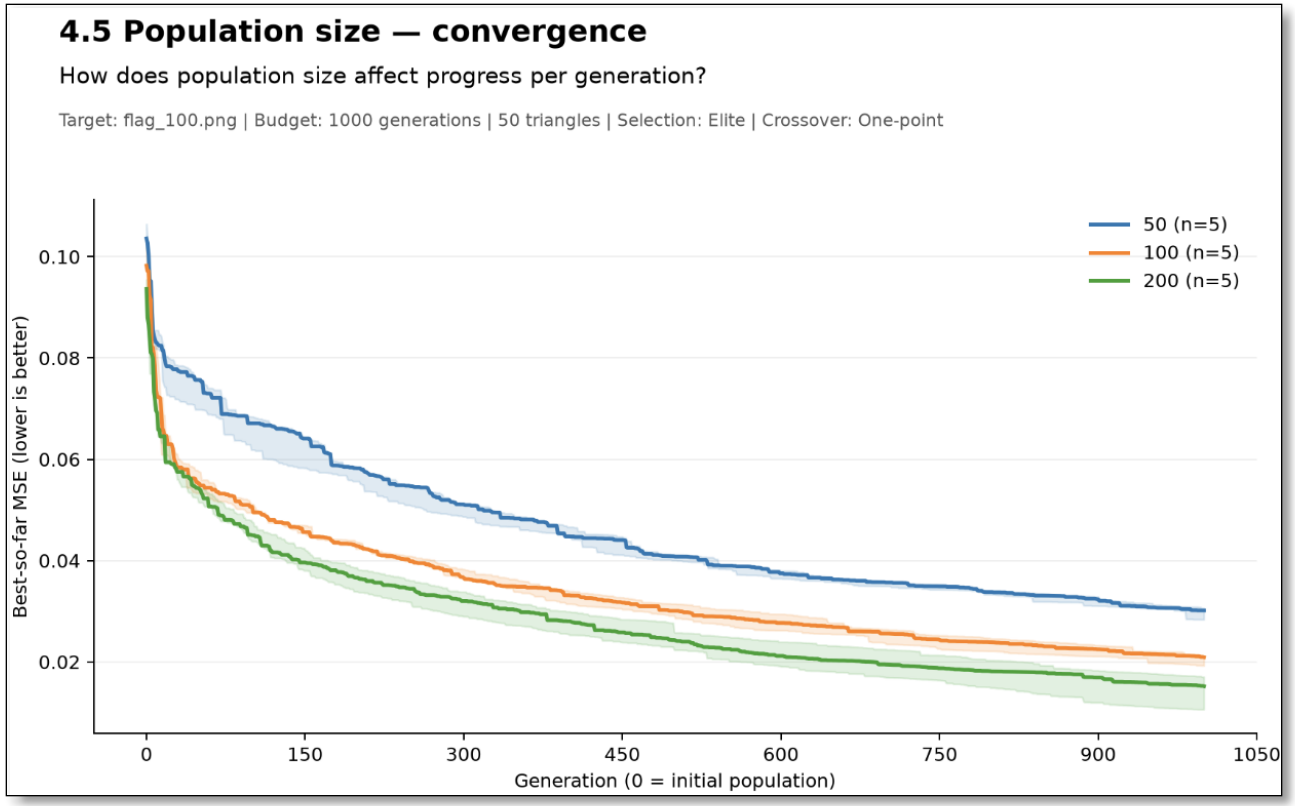
- 20 triangles — MSE 0.0238 · ~12.5 s
- 50 triangles — MSE 0.0302 · ~22.6 s
- 100 triangles — MSE 0.0346 · ~41.3 s

Conclusion: More triangles increased computational cost without improving final MSE under the fixed 1000-generation budget.



POPULATION SIZE - CONVERGENCE:

How does population size affect convergence over a fixed number of generations?

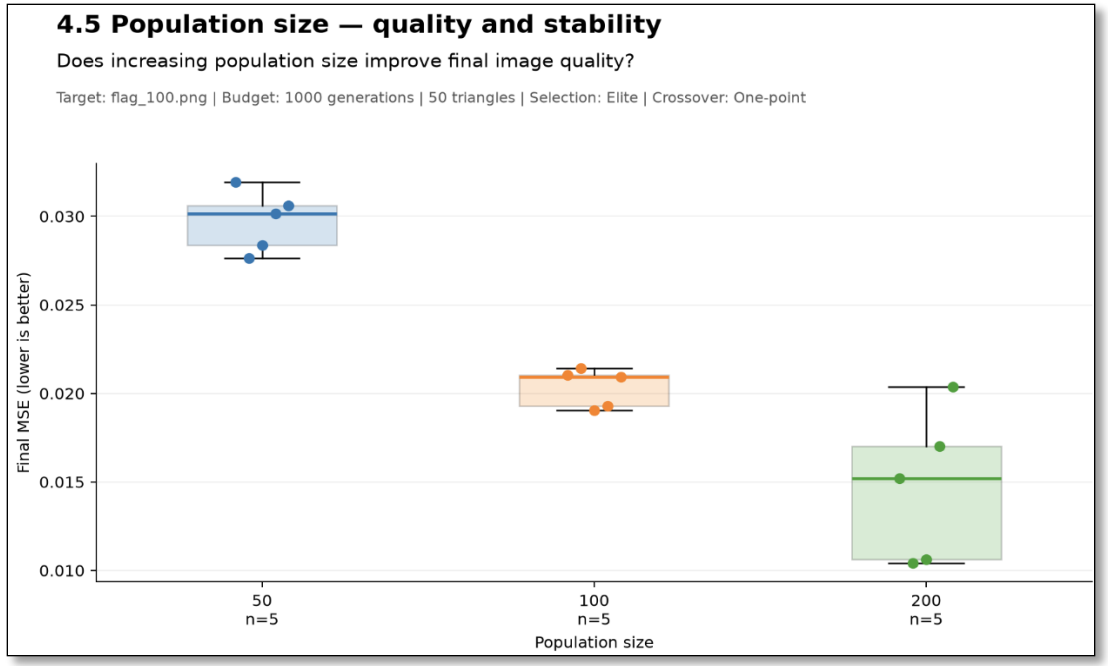


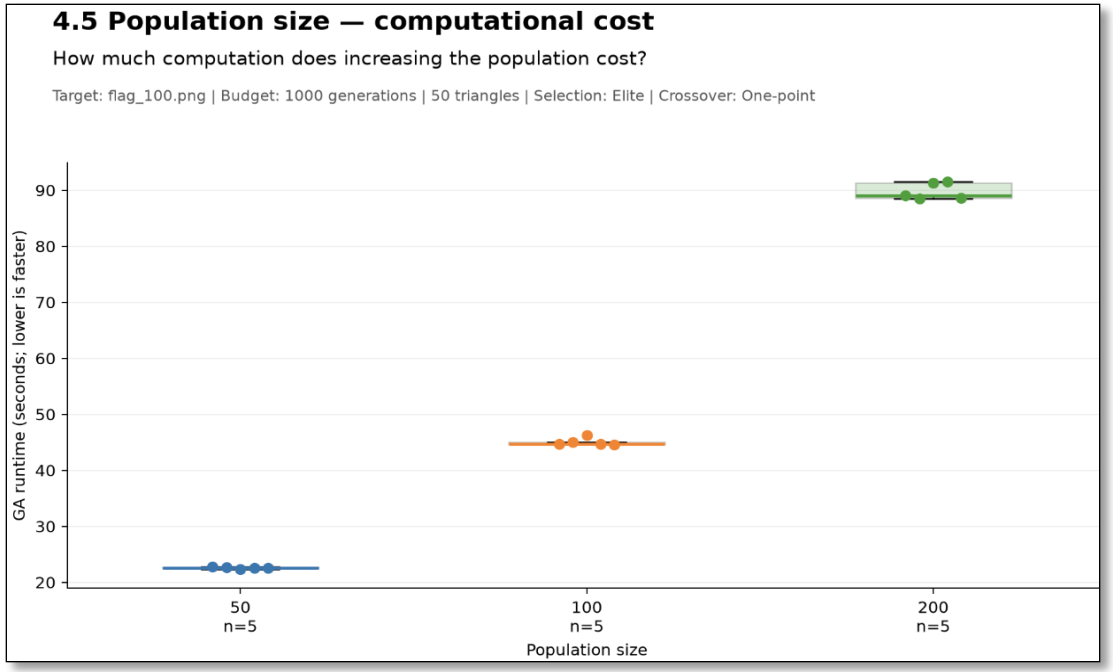
- FASTER IMPROVEMENT PER GENERATION
  - Larger populations reach lower MSE earlier in the run.
- LOWER ERROR THROUGHOUT
  - Population 200 remains below 100 and 50 for almost the entire search.
- NO EARLY PLATEAU
  - All configurations continue improving after 1000 generations.

Larger populations achieve better solutions within the same generation budget. However, this does not necessarily mean they are computationally faster, since each generation contains more individuals."

POPULATION SIZE – QUALITY VS. COMPUTATIONAL COST

How does increasing population size affect the trade-off between approximation quality and computational cost?





POP. 50: Median MSE: 0.0302 (baseline)
POP. 100: Median MSE: 0.0209 (-31%)
POP. 200: Median MSE: 0.0152 (-50%)

POP. 50: ~23 s (baseline)
POP. 100: ~45 s (~2×)
POP. 200: ~89 s (~4×)

Larger populations improve solution quality per generation, but the largest population also shows greater variability across seeds, and runtime increases approximately proportionally with population size.

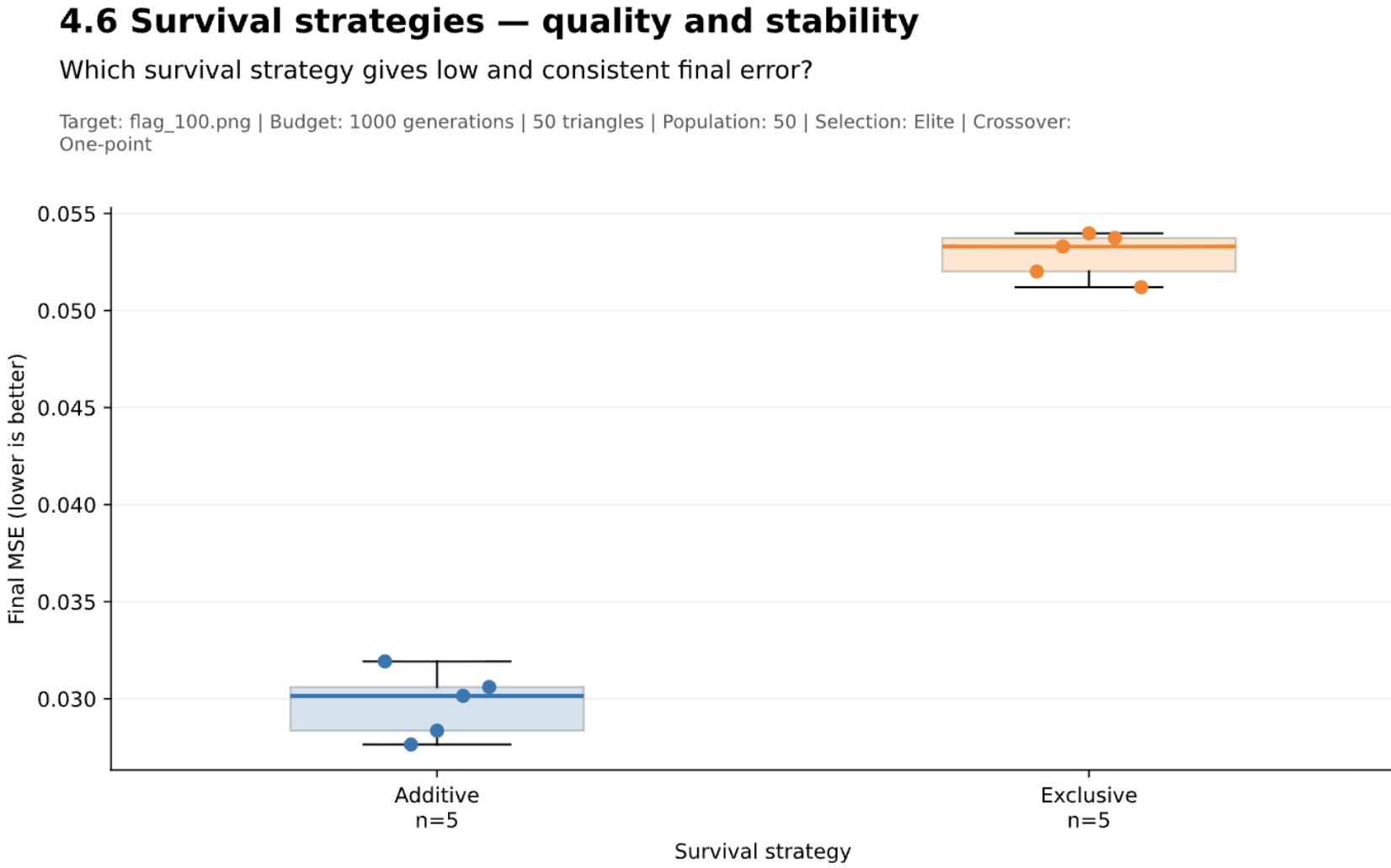
SURVIVAL STRATEGY

How does the survival strategy affect final approximation quality?

Additive — 0.0302
Exclusive — 0.0533

Conclusion: Additive survival achieved clearly lower final error than Exclusive survival.

Additive survival can keep good parents, while Exclusive survival replaces them with offspring.



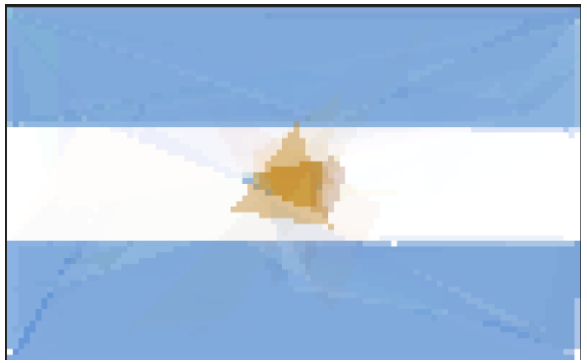
RESULTS SUMMARY

Parameter	Best in individual experiment	Main finding
Selection	Tournament Det.	Lowest median MSE; several methods performed similarly
Crossover	Uniform	Lower error and low variation
Mutation	Multigene / Uniform	Largest observed effect on quality
Triangles	20	Fewer performed better under fixed generation budget
Population	200	Better quality, but ~4× runtime vs. 50
Survival	Additive	Lower error than Exclusive

Overall pattern: Search operators had a strong impact on performance, while increasing problem size or computational effort introduced clear quality–cost trade-offs.

FINAL VALIDATION

TARGET → VALIDATION RESULT



- Promising settings from the controlled experiments were combined in a longer validation run.
- This is a single validation run and is not included in the controlled comparison.

Validation run

Triangles: 50

Population: 100

Generations: 2000

Selection: Deterministic Tournament

Crossover: Uniform

Mutation: Uniform

Survival: Additive

Result

Final MSE: 0.00104

Final fitness: 0.99896

Runtime: ~100 s


```
"triangles": [
  {
    "vertices": [
      [
        0.2577886895967729,
        0.7432981539884098
      ],
      [
        0.28714122380472895,
        0.5051311055940245
      ],
      [
        0.8384752021069515,
        0.3592056019474247
      ]
    ],
    "color": [
      0.23902604648911474,
      0.22140191401177634,
      0.42218680477110926,
      0.4076034334980474
    ]
```


Triangle Enumeration

- Each triangle is stored with three normalized vertex coordinates and RGBA color

CONCLUSIONS

- Conclusions Outsource
  - Mutation strategy had a major impact on performance.
  - Uniform crossover achieved lower error than One-point and Two-point.
  - Additive survival achieved lower error than Exclusive survival.
  - Larger populations improved quality, but increased runtime.
  - More triangles did not improve quality under the fixed search budget.
- Possible improvements Marek
  - Test more random seeds for more reliable results.
  - Explore fitness functions that better measure visual similarity.
  - Use adaptive mutation rates during evolution.
  - Divide high-resolution images into smaller regions and optimize them separately.

Possible improvement: region-based optimization